

PÉCULE

Ton premier patrimoine, sans risque

Cahier des charges personnel

Projet Flutter Investment Companion, module Compilation et Développement d'Applications Mobiles

Élément	Valeur
Auteur	Banconlé AKOBI (projet individuel)
Formation	FISA 5 Informatique, INSA Hauts-de-France, 2026-2027
Version	1.0 du 5 octobre 2026
Proposition validée	Proposition remise le 23 septembre 2026
Budget de développement	Environ 60 heures de code (rapport et soutenance en plus)

Sommaire

Si le sommaire apparaît vide, fais un clic droit dessus puis « Mettre à jour les champs » dans Word.

1. Vue d'ensemble

1.1 À quoi sert ce document

Ce cahier des charges est écrit pour moi seul. Il fixe ce que Pécule doit faire, comment je vais le construire et dans quel ordre, afin que chaque session de travail parte d'une décision déjà prise plutôt que d'une hésitation. Il sert aussi de base au rapport final : les choix techniques y sont déjà justifiés, il suffira de les compléter avec ce qui a réellement été fait.

Les sections marquées **à compléter** ou les cadres **Capture à insérer** sont prévus pour être remplis au fil du projet. Le tableau de traçabilité (section 2) est la référence pour vérifier, avant la soutenance, que chaque exigence de l'énoncé est couverte.

1.2 Le problème

Un débutant de 20 à 30 ans(ou plus) entend parler d'ETF, de DCA ou de crypto sans savoir par où commencer. Les ressources existantes se rangent dans deux familles qui ne lui conviennent pas : les cours théoriques, qui expliquent sans montrer, et les plateformes de trading, qui montrent mais l'exposent immédiatement à de l'argent réel.

Pécule se place entre les deux. L'utilisateur se construit un patrimoine fictif à partir de cours réels, comprend ce qu'il voit grâce à des explications contextualisées, et apprend par l'interaction avec ses propres données plutôt que par la lecture.

1.3 Utilisateur cible

Persona	Description
Profil	Étudiant ou jeune actif de 20 à 30 ans, à l'aise avec les applications mobiles, curieux de l'investissement mais sans connaissances solides.
Habitudes	Utilise déjà des applications au design soigné (Finary, Trade Republic, néobanques). Il attend une interface sombre, épurée, avec des chiffres mis en avant.
Frein principal	La peur de perdre de l'argent sans comprendre pourquoi, et le jargon.
Ce qu'il veut	Comprendre ce que signifie concrètement « +18 % sur un an », tester des stratégies sans risque, et savoir s'il est prêt à passer au réel.

1.4 Promesse et ton

La promesse tient dans le sous-titre : ton premier patrimoine, sans risque. L'application tutoie l'utilisateur, parle simplement et ne juge jamais une question. Chaque chiffre important est accompagné d'une explication accessible en un geste, grâce aux cartes « Ça veut dire quoi ? ».

1.5 Périmètre

Conformément à l'énoncé, Pécule ne permet **aucune transaction financière réelle** et ne demande **aucune connexion à un compte bancaire ou à un courtier**. Il n'y a pas de compte utilisateur ni de synchronisation dans le cloud : toutes les données restent sur le téléphone.

2. Traçabilité des exigences de l'énoncé

Ce tableau relie chaque contrainte imposée (section 11 de l'énoncé) à la manière dont Pécule la satisfait. La colonne Statut se coche au fil du projet ; avant la soutenance, toutes les lignes doivent être cochées.

Exigence de l'énoncé	Réponse dans Pécule	Où	Statut
Au moins 3 écrans significatifs et navigation	Onboarding, 4 onglets (Patrimoine, Explorer, Apprendre, Simulateur), fiche actif, formulaire d'achat, détail de leçon	page 4	☐
Un formulaire avec validation	Formulaire d'achat fictif et formulaire de simulation DCA, avec messages d'erreur explicites	page 4.5, page 4.7	☐
Une liste dynamique	Catalogue de l'onglet Explorer avec recherche, filtres et favoris ; liste des positions du portefeuille	page 4.3	☐
Au moins une API REST et conversion JSON vers objets Dart	Twelve Data, CoinGecko et Frankfurter, avec DTO et méthodes fromJson testées	page 5	☐
Persistance de données métier	SQLite : profil, leçons vues, favoris, transactions, simulations, préférences	page 6	☐
Cache local de données issues d'une API	Historiques de cours et taux de change en base, téléchargement incrémental	page 7	☐
Fonctionnement partiel hors connexion	Tout reste consultable avec la date de dernière mise à jour ; bandeau hors connexion	page 7.4	☐
Gestion explicite des états de chargement, données, erreur	AsyncValue de Riverpod et objet CachedData portant la fraîcheur	page 9	☐
Au moins une visualisation graphique significative	Courbe interactive de la fiche actif, anneau de répartition, double courbe du simulateur	page 4, page 10	☐
Au moins deux calculs métier	Neuf calculs : portefeuille, DCA, effet de change, volatilité, drawdown, score, concentration...	page 8	☐
Au moins trois animations dont deux codées	Quatre animations codées (AnimationController et Tween) et des micro-animations	page 10	☐
Tests unitaires sur la logique métier	Couverture complète de la couche domain, cas limites compris	page 12	☐

L'énoncé insiste aussi sur trois points évalués à part entière : l'architecture (page 12 de l'énoncé, traité en section 11 ici), la stratégie de cache justifiée dans le rapport (section 7) et l'explicabilité du score (section 8.4).

3. Fonctionnalités et priorités

Chaque fonctionnalité porte un identifiant pour pouvoir y faire référence dans les commits, le planning et le rapport. Les priorités suivent trois niveaux.

Priorité	Signification
Indispensable	Exigée par l'énoncé ou au cœur de la promesse. Sans elle, le projet n'est pas rendu.
Important	Rend l'application nettement meilleure. Prévue dans les 60 heures.
Bonus	Uniquement si tout le reste est terminé, testé et propre.

ID	Fonctionnalité	Priorité
F-ONB-01	Questionnaire de premier lancement qui détermine un niveau	Indispensable
F-ONB-02	Possibilité de passer le questionnaire et de le refaire depuis les préférences	Important
F-PAT-01	Valeur totale du portefeuille fictif en euros, gain ou perte, performance	Indispensable
F-PAT-02	Anneau de répartition par type d'actif (actions, ETF, cryptos)	Indispensable
F-PAT-03	Liste des positions avec valeur et performance de chaque ligne	Indispensable
F-PAT-04	Conseil de diversification quand une ligne pèse trop lourd	Important
F-EXP-01	Catalogue d'une trentaine d'instruments avec dernier cours et variation	Indispensable
F-EXP-02	Recherche par nom ou symbole et filtre par type	Indispensable
F-EXP-03	Ajout et retrait des favoris, avec filtre « Mes favoris »	Indispensable
F-FIC-01	Courbe interactive avec choix de période et lecture au doigt	Indispensable
F-FIC-02	Score de stabilité détaillable (volatilité et plus forte baisse)	Indispensable
F-FIC-03	Performance en devise d'origine comparée à la performance en euros	Indispensable
F-FIC-04	Cartes « Ça veut dire quoi ? » sur les indicateurs	Important
F-FIC-05	Bouton d'achat fictif ouvrant le formulaire	Indispensable
F-ACH-01	Formulaire d'achat fictif avec validation	Indispensable
F-APP-01	Liste de leçons courtes, mises en avant selon le niveau	Indispensable
F-APP-02	Leçon illustrée par une donnée réelle avec lien vers une fiche ou le simulateur	Important
F-APP-03	Suivi des leçons vues	Important
F-SIM-01	Simulation DCA sur cours réels (actif, montant, fréquence, date de début)	Indispensable
F-SIM-02	Résultat animé et double courbe capital investi contre valeur	Indispensable
F-SIM-03	Sauvegarde et liste des simulations	Important
F-SIM-04	Comparaison avec un investissement en une seule fois	Bonus
F-GEN-01	Indicateur de fraîcheur des données et bandeau hors connexion	Indispensable

ID	Fonctionnalité	Priorité
F-GEN-02	Préférences : refaire l'onboarding, vider le cache, réinitialiser le portefeuille	Important

4. Écrans, parcours et critères d'acceptation

Pour chaque écran : son rôle, son contenu, ses états et les critères qui permettent de dire qu'il est terminé. Les cadres en pointillés accueilleront les maquettes produites avec les prompts de la section 13.

4.1 Navigation générale

Au premier lancement, l'utilisateur arrive sur l'onboarding. Ensuite, l'application s'ouvre directement sur une barre de navigation à quatre onglets : Patrimoine, Explorer, Apprendre et Simulateur. La fiche actif, le formulaire d'achat et le détail d'une leçon s'ouvrent par-dessus l'onglet courant, de sorte que le retour ramène toujours là où l'utilisateur était. Chaque onglet conserve sa position de défilement quand on en change.

Capture à insérer : schéma de navigation

4.2 Onboarding

Cinq questions courtes à choix unique situent l'utilisateur. Chaque bonne réponse vaut un point ; le total donne un niveau qui sert à ordonner les leçons de l'onglet Apprendre. Le but n'est pas d'évaluer mais d'orienter, et l'écran de fin le dit clairement.

#	Question proposée	Leçon associée si erreur
1	Qu'est-ce qu'un ETF ?	Actions, ETF et cryptos
2	Si un actif fait +50 % puis -50 %, où en es-tu ?	Rendement et performance
3	Qu'est-ce que la volatilité ?	La volatilité
4	Pourquoi diversifier son portefeuille ?	La diversification
5	Investir 50 € chaque mois, ça s'appelle comment ?	Le DCA

Score	Niveau	Effet
0 à 1	Découverte	Toutes les leçons de base sont mises en avant
2 à 3	Initié	Seules les leçons correspondant aux erreurs sont mises en avant
4 à 5	À l'aise	Les leçons avancées (change, drawdown) sont proposées en premier

Critères d'acceptation. L'onboarding n'apparaît qu'une fois ; il peut être passé, auquel cas le niveau Découverte est appliqué ; le niveau et les erreurs sont enregistrés en base ; il peut être refait depuis les préférences.

Capture à insérer : onboarding, question et écran de résultat

4.3 Explorer

Une liste des instruments du catalogue, chacun affiché avec son logo ou ses initiales, son nom, son symbole, son type, son dernier cours connu et sa variation sur la période choisie. Une barre de recherche filtre par nom ou symbole au fil de la frappe ; des puces filtrent par type (Tous, Actions, ETF, Cryptos, Favoris). Toucher une ligne ouvre la fiche actif ; toucher le cœur ajoute ou retire le favori.

États à gérer. Premier chargement sans aucune donnée (squelettes de lignes) ; données disponibles ; données disponibles mais anciennes (date affichée) ; recherche sans résultat (message et bouton pour effacer) ; filtre Favoris vide (message qui invite à ajouter un premier favori).

Critères d'acceptation. La recherche ne déclenche aucun appel réseau ; un favori ajouté apparaît immédiatement dans le filtre et survit à un redémarrage ; la liste est utilisable hors connexion avec les derniers cours connus.

Capture à insérer : Explorer, liste, recherche et filtre favoris

4.4 Fiche actif

C'est l'écran le plus riche et celui qui porte la pédagogie. De haut en bas : l'en-tête (nom, cours en euros, variation colorée), la courbe avec ses périodes (1M, 6M, 1A, 3A, Max), le bloc performance devise d'origine contre euros, le score de stabilité, les cartes « Ça veut dire quoi ? » et le bouton « Ajouter à mon portefeuille ».

- **Courbe.** Elle se dessine de gauche à droite à l'ouverture. En glissant le doigt, un curseur affiche la date et le cours du point touché, et l'en-tête affiche la variation entre le début de la période et ce point.
- **Performance et change.** Deux chiffres côte à côte : « En dollars : +18 % » et « Pour toi, en euros : +12,6 % », avec une phrase qui explique l'écart par l'évolution de l'euro face au dollar. Pour un actif coté en euros, le bloc est masqué.
- **Score de stabilité.** Une jauge qui se remplit, un libellé (Agité, Modéré, Stable) et, au toucher, le détail : les deux sous-scores, leurs valeurs brutes et la formule en mots simples. Une mention rappelle que le score décrit le passé et ne prédit rien.
- **Cartes « Ça veut dire quoi ? ».** Chaque indicateur affiché possède une carte dépliant qui l'explique avec les chiffres de cet actif précis, par exemple : « Sur un an, le cours de cet actif s'est éloigné de sa moyenne d'environ 22 % ; c'est plutôt calme pour une action. »

Critères d'acceptation. La fiche s'ouvre instantanément si l'historique est en cache ; sinon un état de chargement occupe la place de la courbe sans bloquer le reste ; un échec réseau affiche une erreur avec bouton Réessayer, et les données en cache restent visibles s'il y en a.

Capture à insérer : fiche actif, haut de page avec courbe

Capture à insérer : fiche actif, score détaillé et cartes explicatives

4.5 Formulaire d'achat fictif

L'utilisateur choisit la date d'achat et le montant en euros. L'application calcule et affiche avant validation le cours à cette date, le taux de change appliqué et la quantité obtenue, afin qu'il comprenne ce qu'il achète.

Champ	Règle de validation	Message
Montant (€)	Nombre strictement positif, au plus 1 000 000, deux décimales maximum	« Indique un montant supérieur à 0 € »
Date d'achat	Ni dans le futur, ni avant le premier cours disponible	« Pas de cours disponible à cette date »
Date tombant un jour sans cotation	Acceptée : le cours du jour de cotation précédent est utilisé, et c'est affiché	« Cours du vendredi 3 janvier utilisé »

Critères d'acceptation. Le bouton Valider reste désactivé tant que le formulaire est invalide ; la transaction enregistrée contient le cours et le taux utilisés au moment de l'achat, pour que la valeur historique ne change jamais après coup.

Capture à insérer : formulaire d'achat, état valide et état en erreur

4.6 Patrimoine

L'écran d'accueil. En haut, la valeur totale en euros défile jusqu'à son montant, avec en dessous le gain ou la perte en euros et en pourcentage. Vient ensuite l'anneau de répartition par type d'actif, puis la liste des positions. Si une seule ligne dépasse 50 % de la valeur totale, une carte de conseil s'affiche : « Ton patrimoine dépend à 68 % de Bitcoin. Voir la leçon sur la diversification. »

États métier. Portefeuille vide : illustration, phrase d'invitation et bouton vers Explorer. Portefeuille rempli : contenu complet. Valeurs calculées sur des cours anciens : la date de mise à jour apparaît sous le total.

Capture à insérer : Patrimoine, portefeuille rempli

Capture à insérer : Patrimoine, portefeuille vide

4.7 Simulateur

Le formulaire demande l'actif, le montant périodique, la fréquence (chaque semaine ou chaque mois) et la date de début. Le résultat se dévoile progressivement : le capital investi, puis la valeur finale qui défile, puis la performance et le nombre d'achats effectués. Une double courbe montre le capital investi (en escalier) et la valeur du portefeuille simulé au fil du temps. La simulation peut être enregistrée et retrouvée dans une liste sous le formulaire.

Validation. Montant entre 1 € et 10 000 € ; date de début antérieure d'au moins une période à aujourd'hui et postérieure au premier cours disponible ; actif obligatoire.

Capture à insérer : Simulateur, formulaire

Capture à insérer : Simulateur, résultat animé et double courbe

4.8 Apprendre

Une liste de leçons courtes, ordonnées selon le niveau issu de l'onboarding, chacune marquée comme vue ou non. Une leçon tient sur un écran ou deux, et se termine par un appel à l'action vers une donnée réelle : ouvrir une fiche, lancer une simulation pré-remplie ou regarder son propre portefeuille. Le contenu des leçons est stocké dans un fichier JSON embarqué dans l'application, ce qui le rend disponible hors connexion et facile à modifier.

Leçon	Niveau	Illustration par la donnée
Actions, ETF et cryptos	Découverte	Comparer la fiche d'Apple, du S&P 500 et du Bitcoin
Rendement et performance	Découverte	« +18 % sur un an, ça veut dire quoi ? » sur un actif réel
Les intérêts composés	Découverte	1 000 € à 5 % pendant 10 ans, curseur sur la durée
Le DCA	Découverte	Simulation pré-remplie de 50 € par mois depuis 2022
La diversification	Initié	Lien vers l'anneau de son propre portefeuille
La volatilité	Initié	Comparer le score de deux actifs
La plus forte baisse (drawdown)	Initié	Repérer le sommet et le creux sur une courbe
L'effet de change	À l'aise	Performance en dollars et en euros d'une action américaine

Capture à insérer : Apprendre, liste et détail d'une leçon

5. Données externes et API

L'énoncé précise que les offres gratuites évoluent. Chaque point ci-dessous marqué « à vérifier » doit être confirmé dans la documentation officielle avant d'écrire le client correspondant, idéalement avec un test d'apprentissage.

5.1 Fournisseurs retenus

Fournisseur	Usage dans Pécule	Contraintes connues
Twelve Data	Historiques quotidiens OHLCV des actions et ETF (endpoint time_series, intervalle 1day, paramètre start_date pour l'incrémental)	Clé API ; plan gratuit annoncé à 8 crédits par minute et 800 par jour. Couverture des places européennes sur le plan gratuit à vérifier. Cours ajustés ou bruts à vérifier.
CoinGecko (Demo)	Historiques de prix et de capitalisation des cryptos (market_chart/range, devise usd)	Clé Demo envoyée en en-tête. Profondeur d'historique du plan Demo à vérifier : si elle est limitée à un an, le simulateur crypto sera borné en conséquence.
Frankfurter	Taux EUR/USD historiques (série temporelle, base EUR)	Sans clé. Pas de taux les week-ends et jours fériés : on utilise le dernier taux disponible. URL exacte de l'API à vérifier.

5.2 Catalogue initial proposé

Trente instruments, tous cotés en dollars. Ce choix est volontaire : il évite les incertitudes sur les places européennes du plan gratuit et rend l'effet de change visible sur chaque fiche, ce qui sert directement la pédagogie. Le catalogue est une constante du code (identifiant, nom, symbole, type, fournisseur, devise) et non une donnée téléchargée.

Actions (10)	ETF (10)	Cryptos (10, id CoinGecko)
Apple (AAPL)	SPDR S&P 500 (SPY)	Bitcoin (bitcoin)
Microsoft (MSFT)	Invesco Nasdaq 100 (QQQ)	Ethereum (ethereum)
Nvidia (NVDA)	Vanguard Total World (VT)	Solana (solana)
Amazon (AMZN)	Vanguard Developed Markets (VEA)	BNB (binancecoin)
Alphabet (GOOGL)	Vanguard Emerging Markets (VWO)	XRP (ripple)
Meta (META)	Vanguard Total Bond (BND)	Cardano (cardano)
Tesla (TSLA)	SPDR Gold (GLD)	Dogecoin (dogecoin)
Coca-Cola (KO)	Vanguard Real Estate (VNQ)	Avalanche (avalanche-2)
Johnson & Johnson (JNJ)	Schwab US Dividend (SCHD)	Chainlink (chainlink)
Visa (V)	iShares Russell 2000 (IWM)	Polkadot (polkadot)

Ce catalogue est une proposition à **compléter ou modifier**. La règle à garder : chaque type d'actif doit offrir des profils de risque contrastés, pour que le score de stabilité et les comparaisons aient du sens (Coca-Cola contre Tesla, obligations contre Nasdaq, Bitcoin contre Dogecoin).

5.3 Conséquence du quota de Twelve Data

Avec 8 crédits par minute, télécharger l'historique des vingt actions et ETF d'un coup prendrait près de trois minutes. L'application ne le fait donc jamais en bloc. Les historiques sont chargés à la demande, quand une fiche est ouverte, qu'une simulation est lancée ou qu'un actif entre dans le portefeuille. Toutes les requêtes Twelve Data passent par une file d'attente unique qui respecte le débit autorisé, et une erreur de quota est traduite en message clair plutôt qu'en plantage.

5.4 Clés API

Les clés ne sont jamais écrites dans le code ni commitées. Elles sont injectées à la compilation avec `--dart-define-from-file`, à partir d'un fichier exclu par le `.gitignore`, et un fichier d'exemple sans valeurs est versionné. Le rapport mentionnera la limite connue : une clé embarquée dans une application mobile reste extractible ; c'est acceptable pour des clés gratuites dans un projet pédagogique, alors qu'en production on passerait par un serveur intermédiaire.

6. Persistance et modèle de données

6.1 Choix de la solution

Toutes les données sont stockées dans une base SQLite locale, comme annoncé dans la proposition. Pour y accéder, je propose **drift** plutôt que sqflite : les requêtes sont vérifiées à la compilation, les tables sont décrites en Dart, les migrations sont encadrées et la base peut tourner en mémoire dans les tests, ce qui rend les repositories testables sans téléphone. Ce choix ajoute une génération de code (build_runner) ; c'est le prix à payer et il sera justifié dans le rapport.

Choix à valider avant le sprint 2. Alternative plus légère : sqflite avec des requêtes SQL écrites à la main.

6.2 Séparation des données

La base est découpée en deux familles de tables, exactement comme l'énoncé le demande. Les **données utilisateur** sont précieuses : elles ne sont jamais supprimées automatiquement et ne dépendent d'aucun réseau. Les **données de cache** sont reconstructibles : elles peuvent être vidées à tout moment sans rien perdre d'autre qu'un peu de temps de téléchargement.

Données utilisateur

Table	Colonnes principales	Rôle
user_profile	id, level, quiz_mistakes, onboarding_done, created_at	Niveau issu de l'onboarding
lesson_progress	lesson_id, seen_at	Leçons vues
favorite	asset_id, added_at	Favoris de l'Explorer
paper_transaction	id, asset_id, executed_on, amount_eur, unit_price, price_currency, eur_usd_rate, quantity, created_at	Achats fictifs ; le cours et le taux sont figés au moment de l'achat
saved_simulation	id, asset_id, periodic_amount_eur, frequency, start_date, created_at	Paramètres des simulations ; le résultat est recalculé à l'ouverture
preference	key, value	Préférences simples

Données de cache

Table	Colonnes principales	Rôle
price_bar	asset_id, day, open, high, low, close, volume, market_cap (cryptos)	Historique quotidien ; clé primaire (asset_id, day)
fx_rate	day, base, quote, rate	Taux de change quotidiens ; clé primaire (day, base, quote)
sync_state	resource_key, last_data_day, last_fetched_at	Mémoire de synchronisation par ressource (un actif ou une paire de devises)

Le schéma des tables et leurs relations sera illustré ici dans le rapport.

Capture à insérer : schéma de la base de données

7. Stratégie de cache, d'invalidation et d'actualisation

7.1 Principe

Les cours sont quotidiens : une donnée de clôture ne change plus une fois publiée. Le cache peut donc être considéré comme définitif pour le passé, et la seule question à se poser est de savoir s'il manque des jours récents. C'est cette propriété qui rend possible un téléchargement purement incrémental.

7.2 Décision « cache ou réseau »

C'est le repository de chaque ressource qui décide, jamais un widget. Quand on lui demande l'historique d'un actif, il applique les règles suivantes dans l'ordre.

1. Il lit immédiatement ce qu'il a en base et le renvoie à l'interface, si quelque chose existe.
2. Il consulte `sync_state`. Si `last_fetched_at` date de moins de six heures, il s'arrête : les données sont considérées comme fraîches.
3. Sinon, il calcule le dernier jour de cotation attendu (veille ou jour même selon l'heure, en sautant les week-ends pour les actions ; tous les jours pour les cryptos et le change). Si `last_data_day` est déjà ce jour-là, il met à jour `last_fetched_at` et s'arrête.
4. Sinon, il ne demande à l'API que les jours manquants, à partir de `last_data_day` plus un jour, les insère en base, met à jour `sync_state` et renvoie l'historique complété.
5. En cas d'échec réseau ou de quota, il conserve les données en base, les marque comme anciennes et laisse l'erreur remonter sous forme d'information, pas de plantage.

7.3 Invalidation et actualisation

- **Tirer pour actualiser** ignore le délai de six heures mais garde la logique incrémentale.
- **Vider le cache** (préférences) supprime les trois tables de cache et rien d'autre.
- Les cours du jour pour les cryptos, qui bougent en continu, sont remplacés à chaque synchronisation : seule la dernière journée est réécrite, le reste n'est jamais touché.
- Si les cours fournis sont bruts et non ajustés (division d'actions, dividendes), un écart peut apparaître sur la performance d'un actif concerné. Cette limite est documentée dans le rapport, comme l'annexe A.5 de l'énoncé le demande.

7.4 Fonctionnement hors connexion

Hors connexion, tout ce qui a déjà été vu reste consultable : favoris, portefeuille, leçons, simulations et historiques téléchargés. Chaque valeur issue du réseau est accompagnée de sa date de mise à jour (« Mis à jour le 4 oct. à 18:02 »), et un bandeau discret signale l'absence de réseau. Un actif dont l'historique n'a jamais été téléchargé affiche un état vide explicite plutôt qu'une erreur.

Scénario de la soutenance à reproduire avant le rendu : ouvrir plusieurs fiches et le portefeuille, fermer complètement l'application, couper le réseau, rouvrir et parcourir les quatre onglets.

8. Calculs métier

Tous les calculs sont des fonctions pures écrites en Dart sans aucune dépendance à Flutter. Elles prennent des données en entrée, renvoient un résultat et ne touchent ni au réseau ni à la base. C'est ce qui les rend simples à tester et c'est la réponse à la question de l'énoncé « où se trouvent les calculs métier ? ».

8.1 Conventions

- Les cours stockés sont en devise d'origine. Le taux Frankfurter r (base EUR) indique combien de dollars vaut un euro ; un prix en euros vaut donc prix en dollars divisé par r .
- Pour une date sans cotation ou sans taux, on prend la dernière valeur disponible avant cette date.
- Un calcul impossible (portefeuille vide, historique trop court) ne renvoie jamais 0 par défaut ni null : il renvoie un résultat explicitement indisponible, que l'interface sait afficher.

8.2 Portefeuille

Calcul	Formule	Cas limites à tester
Quantité achetée	$\text{quantité} = \text{montant_eur} \times r_{\text{achat}} / \text{cours_achat}$	Montant nul ; cours nul ; actif coté en euros ($r = 1$)
Valeur d'une ligne	$\text{valeur_eur} = \text{quantité} \times \text{cours_actuel} / r_{\text{actuel}}$	Cours actuel manquant
Valeur totale	$\Sigma \text{valeur des lignes}$	Portefeuille vide
Gain ou perte	$\text{gain} = \text{valeur totale} - \text{montant total investi}$	Gain négatif
Performance	$\text{perf} = \text{gain} / \text{montant investi}$	Montant investi nul (division par zéro)
Répartition	$\text{poids} = \text{valeur d'un type} / \text{valeur totale}$	Un seul type ; somme des poids égale à 1
Concentration	alerte si le poids d'une ligne > 50 %	Exactement 50 % ; une seule ligne

8.3 Simulation DCA

Pour chaque date prévue entre la date de début et aujourd'hui, selon la fréquence choisie, on prend le premier cours de clôture disponible à partir de cette date, on le convertit en euros avec le taux du jour, puis on calcule la quantité achetée : montant périodique divisé par le cours en euros. Les quantités sont additionnées puis valorisées au dernier cours connu.

Sortie	Formule
Capital investi	$\text{nombre d'achats} \times \text{montant périodique}$
Valeur finale	$\Sigma \text{quantités} \times \text{cours final en euros}$
Performance	$\text{valeur finale} / \text{capital investi} - 1$
Prix moyen d'achat	$\text{capital investi} / \Sigma \text{quantités}$
Série pour la courbe	pour chaque jour : capital investi cumulé et valeur des quantités cumulées

Cas limites : date de début dans le futur, aucun cours après la date de début, historique avec des trous, montant nul, simulation d'une seule période, dernier achat tombant aujourd'hui.

8.4 Effet de change

La performance en euros se déduit de la performance en dollars et de l'évolution du taux : $\text{perf_eur} = (1 + \text{perf_usd}) \times r_{\text{début}} / r_{\text{fin}} - 1$. L'effet de change est l'écart entre les deux, en points. Exemple de test et d'affichage : une action gagne 18 % en dollars pendant que l'euro passe de 1,05 à 1,10 dollar ; pour un investisseur européen, la performance n'est plus que de 12,6 %.

8.5 Risque et score de stabilité

Indicateur	Définition	Cas limites
Rendements quotidiens	$r_t = \text{cours}_t / \text{cours}_{(t-1)} - 1$	Moins de deux cours
Volatilité annualisée	écart-type des rendements $\times \sqrt{N}$, avec $N = 252$ pour actions et ETF et $N = 365$ pour les cryptos	Moins de deux rendements ; série constante (volatilité nulle)
Maximum drawdown	plus forte baisse entre un sommet et le creux qui le suit : $\min(\text{cours} / \text{plus haut précédent} - 1)$	Série croissante (0 %) ; exemple de l'annexe A.2 : -28 %

Le score de stabilité est calculé sur un an glissant et combine les deux indicateurs à parts égales. Les plafonds de 80 % servent à normaliser : au-delà, l'actif est considéré comme maximalement agité.

```
note_volatilité = 100 × (1 - min(volatilité, 0,80) / 0,80)
note_baisse     = 100 × (1 - min(|drawdown|, 0,80) / 0,80)
score           = arrondi(0,5 × note_volatilité + 0,5 × note_baisse)
```

0 à 39 : Agité 40 à 69 : Modéré 70 à 100 : Stable

Exemple : une volatilité de 20 % et une plus forte baisse de 25 % donnent 75 et 68,75, soit un score de 72, « Stable ». L'interface affiche toujours ce détail sur demande et rappelle que le score est un repère pédagogique construit sur le passé, pas une prédiction.

8.6 Intérêts composés (leçon)

$\text{capital final} = \text{capital initial} \times (1 + \text{taux})^{\text{années}}$. Utilisé dans la leçon dédiée, avec un curseur sur la durée. Exemple de test : 1 000 € à 5 % pendant 10 ans donnent environ 1 628,89 €.

9. Gestion des états

9.1 Choix de Riverpod

Riverpod est retenu parce qu'il répond à trois besoins du projet sans complexité excessive. Il représente nativement un traitement asynchrone sous la forme d'un AsyncValue (chargement, données, erreur), il permet d'injecter les repositories et de les remplacer par des doublures dans les tests, et il garde l'état hors des widgets, ce qui évite de perdre des données quand un widget est reconstruit. Les providers écoutés par l'écran sont libérés automatiquement quand l'écran disparaît.

9.2 États asynchrones

État	Ce que voit l'utilisateur
Initial	Rien n'a encore été demandé ; ne dure qu'un instant
Chargement sans données	Squelettes animés à l'emplacement exact du contenu
Données disponibles	Contenu complet avec la date de mise à jour
Actualisation	Contenu actuel conservé et indicateur discret ; jamais d'écran vide pendant un rafraîchissement
Erreur sans données	Message compréhensible et bouton Réessayer
Erreur avec données en cache	Contenu en cache, date affichée et bandeau d'information

Pour porter la fraîcheur jusqu'à l'interface, les repositories renvoient un objet CachedData contenant la valeur, la date de dernière mise à jour et l'indication que la donnée est peut-être ancienne. L'interface n'a ainsi jamais à deviner d'où vient ce qu'elle affiche.

9.3 États métier

- Premier lancement ou non (onboarding fait).
- Leçon vue ou non.
- Portefeuille vide ou rempli ; alerte de concentration active ou non.
- Actif en favori ou non.
- Simulation non configurée, configurée, calculée, enregistrée.
- Mode connecté ou hors connexion.

10. Animations et visualisations

L'énoncé demande au moins trois animations dont deux codées. Pécule en prévoit quatre codées avec AnimationController et Tween, chacune justifiée par un usage, plus des micro-animations.

Animation	Type	Technique	Intérêt pour l'utilisateur
Le patrimoine qui défile jusqu'à sa valeur	Codée	AnimationController et Tween<double>, chiffres à chasse fixe pour éviter les sauts	Attire l'œil sur le chiffre principal et rend le gain tangible
La courbe qui se dessine à l'ouverture d'une fiche	Codée	CustomPainter et extraction progressive du tracé (PathMetric)	Fait lire l'histoire du cours dans le bon sens, de gauche à droite
Le score de stabilité qui se remplit	Codée	AnimationController, CurvedAnimation et jauge dessinée	Montre le score comme une mesure, pas comme un verdict
Le résultat de simulation qui se dévoile	Codée	Animation échelonnée (Interval) sur un seul controller	Raconte le résultat dans l'ordre : investi, obtenu, performance
Cœur qui bat à l'ajout d'un favori	Micro	AnimatedScale ou TweenAnimationBuilder	Confirme l'action sans message
Transitions entre écrans et squelettes de chargement	Micro	Transitions de page et opacité animée	Évite les à-coups
Anneau de répartition	Package ou codée	fl_chart ou CustomPainter (à décider)	Lecture immédiate de la diversification

Chaque AnimationController est créé dans initState et libéré dans dispose ; les animations ne se rejouent pas à chaque reconstruction, seulement quand la valeur cible change.

11. Architecture logicielle

11.1 Couches

Le code est séparé en couches qui ne se connaissent que dans un sens : l'interface dépend de l'état, l'état dépend des repositories et du domaine, les repositories dépendent des sources de données. Le domaine ne dépend de rien.

Couche	Contenu	Ne doit pas
domain	Modèles métier immuables et calculs purs	Importer Flutter, Riverpod, drift ou http
data	Clients API, DTO et fromJson, base drift, repositories qui choisissent entre cache et réseau	Contenir de logique d'affichage
application (providers)	État de chaque écran, orchestration des repositories	Construire des widgets
presentation	Écrans et widgets	Appeler une API, lire la base ou faire un calcul métier

11.2 Arborescence proposée

```
lib/  
  main.dart  
  app/           thème, routeur, configuration  
  core/          formatage (€, %, dates), erreurs, horloge injectable  
  domain/  
    models/      Asset, PriceBar, PaperTransaction, Simulation...  
    calculations/ portfolio, dca, currency, risk, stability_score  
  data/  
    remote/      twelve_data_client, coingecko_client, frankfurter_client, dto/  
    local/       database.dart, tables/, daos/  
    repositories/ price_history_repository, portfolio_repository...  
  features/  
    onboarding/  patrimoine/ explorer/ asset_detail/  
    learn/       simulator/ settings/  
    chacun : screen, widgets/, providers  
  test/          même arborescence que lib/  
  assets/lessons/ contenu des leçons en JSON
```

11.3 Réponses aux questions de l'énoncé

Question	Réponse
Où se trouve l'état et pourquoi ?	Dans les providers Riverpod, hors des widgets, pour qu'il survive aux reconstructions et soit testable
Qui décide d'interroger l'API ou d'utiliser le cache ?	Le repository de la ressource, selon les règles de la section 7
Où se trouvent les calculs métier ?	Dans domain/calculations, en fonctions pures
Qui est responsable de la persistance ?	Les DAO drift, appelés uniquement par les repositories
Comment remontent erreurs et chargements ?	Exceptions typées dans data, converties en AsyncValue par les providers, affichées par des widgets d'état communs

Question	Réponse
Que se passe-t-il quand un widget est reconstruit ?	Il relit l'état existant ; aucun appel réseau n'est relancé et les animations ne repartent pas

11.4 Dépendances envisagées

Paquet	Usage	Statut
flutter_riverpod	Gestion des états	Retenu
drift et sq_lite3_flutter_libs	Base SQLite typée	À valider (section 6)
http	Appels REST	Retenu
go_router	Navigations avec onglets conservant leur état	À valider
intl	Formats français des nombres, devises et dates	Retenu
fl_chart	Anneau de répartition, éventuellement double courbe du simulateur	Optionnel
mocktail	Doublures dans les tests de repositories	Retenu (dev)

Toute dépendance non listée ici doit être justifiée et ajoutée à ce tableau avant d'être installée.

Capture à insérer : schéma des composants et des classes principales

12. Principes de code et qualité

Ces règles reprennent le fichier CLAUDE.md construit à partir de Clean Code de Robert C. Martin, adaptées à Flutter et à ce projet. Elles s'appliquent à mon code comme à celui produit avec Claude Code. Chaque principe est d'abord défini, puis traduit en règle concrète.

12.1 Développement piloté par les tests (TDD)

Définition. On écrit d'abord un test qui décrit un comportement et qui échoue, puis le code minimal qui le fait passer, puis on améliore le code sans changer son comportement. Le cycle s'appelle rouge, vert, refactor.

- Aucune ligne de logique métier sans un test qui a d'abord échoué, et pour la bonne raison.
- Pour corriger un bug : un test qui reproduit le bug d'abord, la correction ensuite.
- Ne jamais désactiver ou affaiblir un test pour faire passer la suite.
- Avant d'intégrer une API, écrire un test d'apprentissage qui vérifie qu'on a bien compris sa réponse, à partir d'un exemple de JSON réel enregistré dans test/fixtures.

12.2 Tests propres : F.I.R.S.T.

Définition. Un bon test est rapide (Fast), indépendant des autres (Independent), reproductible partout (Repeatable), auto-validant parce qu'il réussit ou échoue sans lecture de logs (Self-validating), et écrit juste avant le code (Timely).

- Structure en trois temps : préparer, agir, vérifier.
- Un seul concept par test, dont le nom décrit le comportement attendu, par exemple « renvoie une performance indisponible quand rien n'a été investi ».
- Aucun réseau, aucune horloge réelle, aucun hasard : la date du jour est injectée.
- Les cas limites de la section 8 sont tous testés.

12.3 Noms qui révèlent l'intention

Définition. Un nom doit dire pourquoi la chose existe, ce qu'elle fait et comment l'utiliser, sans qu'un commentaire soit nécessaire.

- Code en anglais, textes affichés en français.
- Classes et variables sont des noms (PaperTransaction), fonctions et méthodes sont des verbes (computeMaxDrawdown).
- Pas d'abréviations obscures, pas de préfixes inutiles, pas de mots bruit comme data, info ou manager.
- Un même concept porte le même mot partout : si on dit fetch dans un client, on ne dit pas get ou retrieve dans l'autre.

12.4 Fonctions courtes qui font une seule chose

Définition. Une fonction doit faire une seule chose, la faire bien, et se lire comme un paragraphe de haut en bas, à un seul niveau d'abstraction.

- Deux ou trois paramètres au plus ; au-delà, un objet paramètre nommé.
- Pas de booléen qui fait faire deux choses différentes à une fonction : on écrit deux fonctions.
- Pas d'effet de bord caché : une fonction qui calcule ne sauvegarde pas.

12.5 Commentaires

Définition. Un commentaire compense un code qui n'a pas réussi à exprimer son intention. Avant d'en écrire un, on essaie de renommer, d'extraire une fonction ou d'introduire une variable explicative.

Autorisé	Interdit
Le pourquoi d'une décision qui n'est pas évidente	Paraphraser le code
L'avertissement d'une conséquence	Raconter l'historique (« modifié pour... », « nouvelle version »)
Le format imposé par une API externe	Laisser du code commenté
La documentation d'une API publique réellement réutilisée	Bannières de section, auteurs, dates, humeurs

12.6 Gestion des erreurs

Définition. Le traitement des erreurs est une responsabilité à part entière, qui ne doit pas noyer la logique principale.

- Exceptions typées et porteuses de contexte (NetworkException, RateLimitException, MissingPriceException).
- Ne pas renvoyer null pour signaler un échec ; un résultat indisponible est un type explicite.
- Le code tiers (http, drift, fl_chart) est enveloppé derrière nos propres classes, pour pouvoir le remplacer ou le simuler.

12.7 Principes de conception

Principe	Définition	Application dans Pécule
Responsabilité unique (SRP)	Une classe n'a qu'une seule raison de changer	Un client API ne parse que son JSON ; un repository ne décide que du cache
DRY	Chaque connaissance a une seule représentation dans le système	Un seul formateur d'euros et de pourcentages, un seul widget d'état d'erreur
KISS	La solution la plus simple qui fonctionne est la meilleure	Pas de couche ni de patron ajoutés par principe
YAGNI	On ne construit pas ce dont on n'a pas besoin maintenant	Pas de multi-devises ni de comptes utilisateurs
Séparer construction et utilisation	Les objets reçoivent leurs dépendances au lieu de les créer	Repositories et horloge fournis par des providers
Loi de Déméter	Un objet ne parle qu'à ses voisins immédiats	Pas de chaînes du type a.b.c.d dans les widgets
Règle du scout	Laisser le code un peu plus propre qu'on ne l'a trouvé	Limitée aux fichiers touchés par la tâche en cours

En cas de conflit entre règles, l'ordre de priorité est celui de la conception simple de Kent Beck : tous les tests passent, pas de duplication, le code exprime clairement l'intention, et enfin le moins de classes et de fonctions possible.

12.8 Règles propres à Flutter

- La méthode build ne fait qu'afficher : aucun appel réseau, aucun accès base, aucun calcul métier.
- Des widgets petits et extraits en classes plutôt qu'en méthodes qui renvoient des widgets ; constructeurs const partout où c'est possible.
- Tout controller (animation, texte, défilement) créé dans initState est libéré dans dispose.
- Aucune couleur ni taille en dur dans les écrans : tout passe par le thème.
- flutter analyze sans aucun avertissement avant chaque commit.

12.9 Travail avec Claude Code

Le fichier CLAUDE.md est placé à la racine du dépôt, avec sa section 0 remplie pour Pécule. Chaque demande à Claude Code porte sur une seule fonctionnalité identifiée (par exemple F-SIM-01) et cite la section concernée de ce cahier des charges. Claude Code doit demander mon accord avant d'ajouter une couche, un patron ou une dépendance, et prouver dans son compte rendu que les tests ont été lancés.

```
## 0. Contexte du projet
- Stack : Flutter (Dart), Riverpod, drift (SQLite), http
- Lancer toute la suite de tests : flutter test
- Build : flutter build apk --dart-define-from-file=env.json
- Lint / format : flutter analyze && dart format .
- Langue : noms et commentaires en anglais, textes affichés en français (tutoiement)
- Particularités : domain/ sans import Flutter ; aucun appel réseau dans build ;
  référence fonctionnelle = cahier des charges Pécule, citer l'ID (F-XXX-00)
```

Une habitude personnelle. Je relis et comprends chaque morceau de code généré avant de le commiter. Lors de la maintenance, on pourra me demander de modifier du code en direct : rien ne doit être dans le dépôt que je ne saurais pas expliquer.

13. Maquettes : prompts à utiliser

Ces prompts sont prévus pour un outil de maquettage par IA. On commence toujours par coller le brief de style, puis le prompt de l'écran voulu dans la même conversation, pour que tous les écrans partagent la même identité. Les captures obtenues vont dans les cadres de la section 4.

13.1 Charte graphique de départ

Élément	Valeur proposée (à ajuster)
Fond	#0B0C0F
Surface (cartes)	#16181D, surface élevée #1F2229, bordures #2A2E36
Texte	principal #F5F6F8, secondaire #9BA1AB
Accent Pécule	#8B7CFF (violet)
Gain / perte	#34D399 / #F87171, toujours accompagnés d'un signe et d'une flèche pour ne pas dépendre de la couleur
Typographie	Inter, chiffres à chasse fixe ; gros chiffres en 40 à 48 pt semi-gras
Formes	Rayons de 16 à 20 px, espacements multiples de 8
Formats	Français : 1 234,56 €, +12,6 %, 4 oct. 2026

Prompt : brief de style (à coller en premier)

Tu es designer produit senior spécialisé en applications fintech mobiles. Je conçois Pécule, une application Flutter (iPhone, 390 x 844) qui aide les débutants de 20 à 30 ans à comprendre l'investissement avec un portefeuille fictif basé sur des cours réels. Aucune transaction réelle.

Univers visuel : proche de Finary et Trade Republic. Mode sombre uniquement, interface très épurée, beaucoup d'espace, chiffres mis en avant en grand. Fond #0B0C0F, cartes #16181D, bordures fines #2A2E36, texte #F5F6F8, texte secondaire #9BA1AB, accent violet #8B7CFF, gain #34D399, perte #F87171. Police Inter, chiffres à chasse fixe. Rayons 16 à 20 px, grille de 8 px. Ton : tutoiement, phrases courtes, jamais de jargon sans explication. Formats français : 1 234,56 €, +12,6 %. Navigation : barre inférieure à 4 onglets avec icônes fines : Patrimoine, Explorer, Apprendre, Simulateur. Confirme que tu as compris, je t'envoie ensuite les écrans un par un.

Prompt : Onboarding

Écran d'onboarding de Pécule, en 3 états côte à côte :

- 1) Accueil : logo Pécule, titre « Ton premier patrimoine, sans risque », sous-titre rassurant, bouton principal « C'est parti » et lien « Passer ».
- 2) Question 2 sur 5 : barre de progression fine en haut, question « Qu'est-ce qu'un ETF ? », 4 réponses en grandes cartes sélectionnables, une carte sélectionnée avec bordure violette, bouton « Suivant ».
- 3) Résultat : niveau « Initié » mis en avant, phrase qui explique que ce n'est pas une note mais un point de départ, liste de 3 leçons conseillées, bouton « Découvrir Pécule ».

Prompt : Patrimoine

Écran Patrimoine (onglet actif), portefeuille rempli :

En haut, petit libellé « Ton patrimoine fictif », valeur totale 3 482,17 €

en très grand, dessous +482,17 € (+16,1 %) en vert avec flèche.
Petite mention grise « Mis à jour le 4 oct. à 18:02 ».
Carte avec anneau de répartition : ETF 55 %, Actions 30 %, Cryptos 15 %, légende à droite. Carte conseil discrète avec icône ampoule : « Ton patrimoine est bien réparti entre 3 types d'actifs. »
Liste « Mes positions » : 5 lignes (logo rond, nom, quantité, valeur en €, performance colorée). Montre aussi une variante « portefeuille vide » avec illustration sobre, phrase « Ton patrimoine commence ici » et bouton « Explorer les actifs ».

Prompt : Explorer

Écran Explorer : barre de recherche arrondie en haut, puis rangée de puces filtres : Tous (actif), Actions, ETF, Cryptos, Favoris (avec cœur).
Liste de 8 actifs : logo rond ou initiales, nom en gras, symbole et type en gris, à droite cours en € et variation 1 an colorée, cœur à droite (plein pour 2 actifs favoris). Montre aussi l'état « aucun résultat » pour une recherche, et l'état de chargement avec lignes squelettes.

Prompt : Fiche actif

Écran Fiche actif pour « Apple (AAPL) », défilable, en 2 captures :
Haut : bouton retour et cœur, nom, cours 214,30 € en grand, variation +12,6 % sur 1 an. Grande courbe lisse sans axes visibles, avec un curseur vertical et une bulle date / cours là où le doigt touche. Sélecteur de période en pastilles : 1M 6M 1A 3A Max.
Bloc « Dollar ou euro ? » : deux chiffres côte à côte, « En dollars +18 % » et « Pour toi, en euros +12,6 % », avec une phrase d'explication courte.
Bas : carte « Score de stabilité » avec jauge en arc remplie à 72/100, libellé « Stable », lien « Comprendre ce score » qui déploie deux sous-scores (volatilité 75, plus forte baisse 69). Mention discrète : « Basé sur le passé, ce n'est pas une prédiction. »
Cartes repliables « Ça veut dire quoi ? ». Bouton principal fixe en bas : « Ajouter à mon portefeuille ».

Prompt : Formulaire d'achat fictif

Feuille modale montante « Achat fictif : Apple ». Champs : montant en € (grand champ numérique centré) et date d'achat (sélecteur).
Encadré récapitulatif qui se met à jour : cours à cette date, taux EUR/USD appliqué, quantité obtenue. Rappel « Aucun argent réel n'est utilisé ».
Montre l'état valide (bouton violet actif) et un état en erreur (montant 0 €, bordure rouge, message « Indique un montant supérieur à 0 € », bouton désactivé).

Prompt : Simulateur

Écran Simulateur, en 2 états :
1) Formulaire : titre « Et si j'avais investi... ? », sélection de l'actif (carte avec logo), montant périodique 50 €, choix Chaque semaine / Chaque mois en segmented control, date de début janv. 2022, bouton « Lancer la simulation ». Sous le formulaire, liste « Mes simulations enregistrées » avec 2 éléments.
2) Résultat : trois chiffres révélés l'un après l'autre : « Tu aurais investi 2 250 € », « Ça vaudrait aujourd'hui 3 104 € », « +38 % ».
Graphique à deux courbes : capital investi en escalier gris, valeur en violet. Boutons « Enregistrer » et « Modifier ».

Prompt : Apprendre

Écran Apprendre : en-tête « Ton parcours », badge niveau « Initié », barre de progression 3 leçons sur 8. Section « Pour toi » avec 2 grandes cartes de leçons mises en avant, puis liste de toutes les leçons avec durée de lecture et coche verte pour les leçons vues.
Ajoute l'écran détail d'une leçon « La volatilité » : texte court aéré, un mini graphique comparant deux courbes (calme contre agitée), et un

bouton final « Voir le score de Tesla ».

Prompt : États transverses

Planche de composants d'état pour Pécule : bandeau hors connexion discret en haut d'écran (« Hors connexion, données du 4 oct. à 18:02 »), carte d'erreur avec icône, message simple et bouton « Réessayer », squelettes de chargement d'une ligne de liste et d'une courbe, et toast de confirmation « Ajouté à tes favoris ».

Capture à insérer : planche des états transverses

14. Étapes et planning

Le projet est découpé en neuf sprints qui totalisent les 60 heures de code. L'ordre n'est pas arbitraire : le domaine et les données viennent avant les écrans, pour que chaque écran soit construit sur des fondations déjà testées. Les dates sont à remplir selon mon alternance entre semaines d'école et semaines en entreprise.

Sprint	Contenu	Heures	Terminé quand...	Dates
S0. Fondations	Création du projet, lints stricts, arborescence, thème sombre, navigation à 4 onglets vides, CLAUDE.md, clés API hors dépôt	4 h	L'application se lance avec ses 4 onglets et flutter analyze est vide	
S1. Domaine	Modèles et tous les calculs de la section 8 en TDD	8 h	Tous les calculs et cas limites sont testés et verts	
S2. Données	Tests d'apprentissage des 3 API, clients et DTO, base drift, repositories, cache incrémental, file d'attente du quota	12 h	Un historique se télécharge, se met en cache, se complète et se lit hors connexion, le tout testé	
S3. Explorer et fiche	Catalogue, recherche, filtres, favoris, fiche actif complète avec courbe et score	10 h	F-EXP et F-FIC terminées avec leurs états	
S4. Portefeuille	Formulaire d'achat validé, écran Patrimoine, anneau, alerte de concentration	8 h	On peut acheter, voir son patrimoine évoluer et redémarrer sans rien perdre	
S5. Simulateur	Formulaire, calcul, double courbe, sauvegarde	6 h	F-SIM-01 à 03 terminées	
S6. Onboarding et Apprendre	Questionnaire, niveau, leçons JSON, ordre selon le niveau, suivi	5 h	Un nouvel utilisateur suit le parcours complet	
S7. Animations et finitions	Les 4 animations codées, micro-animations, cohérence visuelle avec les maquettes	5 h	Les animations sont fluides et libérées correctement	
S8. Recette	Scénario hors connexion, cas limites, nettoyage, captures pour le rapport	2 h	Toute la section 15 est cochée	

Règle de pilotage. Si un sprint dépasse son budget de plus de 30 %, je retire d'abord une fonctionnalité Bonus, puis une Importante, jamais une Indispensable. Chaque sprint se termine par un commit propre et une ligne dans le journal des décisions (section 17).

14.1 Après le développement

Livrable	Contenu	Dates
Rapport	Reprendre ce cahier des charges, compléter ce qui a réellement été fait, ajouter captures, schémas, tests, difficultés et améliorations	
Soutenance	Démonstration suivie d'une discussion technique ; préparer les réponses de la section 11.3 et savoir modifier le code en direct	

15. Recette avant le rendu

Liste à dérouler sur un vrai téléphone, application installée en mode release.

Scénario	Résultat attendu	OK
Premier lancement	L'onboarding apparaît une seule fois	☐
Ouvrir trois fiches puis couper le réseau et redémarrer	Les fiches s'affichent avec leur date de mise à jour et le bandeau hors connexion	☐
Ouvrir une fiche jamais vue hors connexion	État vide explicite, aucun plantage	☐
Acheter un actif à une date de week-end	Cours du dernier jour de cotation utilisé et affiché	☐
Saisir un montant de 0 €	Message d'erreur, bouton désactivé	☐
Simuler depuis une date dans le futur	Validation refusée	☐
Rouvrir l'application le lendemain	Seuls les jours manquants sont téléchargés (vérifiable dans les logs de débogage)	☐
Ajouter 70 % du portefeuille sur un seul actif	Alerte de concentration avec lien vers la leçon	☐
Vider le cache	Portefeuille, favoris et simulations intacts	☐
Faire tourner l'écran et revenir sur un onglet	Aucune perte d'état, aucune animation rejouée à tort	☐
Lancer flutter test et flutter analyze	Tout est vert, aucun avertissement	☐

16. Risques et parades

Risque	Effet	Parade
Quota Twelve Data atteint	Données qui ne se chargent plus pendant la démo	Chargement à la demande, file d'attente, cache préchauffé avant la soutenance
Historique crypto limité sur le plan Demo	Simulateur crypto incomplet depuis 2022	Vérifier tôt ; borner la date de début et l'expliquer dans l'interface
Cours non ajustés	Performance faussée pour un actif qui a divisé son action	Vérifier l'option d'ajustement ; sinon documenter la limite (annexe A.5)
Dépassement du temps	Fonctionnalités non terminées	Priorités strictes et règle de pilotage de la section 14
Code généré mal compris	Difficulté en soutenance	Relecture systématique, CLAUDE.md, petites demandes ciblées
Fuite de clé API	Clé révoquée ou abusée	Clés hors dépôt, fichier d'exemple versionné, vérification avant chaque push

16.1 Hors périmètre

Ne seront pas réalisés, sauf si tout le reste est terminé : comptes utilisateurs et synchronisation, notifications, ventes fictives, frais de courtage simulés, actifs cotés en euros, mode clair, comparaison DCA contre investissement unique (F-SIM-04).

17. Journal des décisions

Une ligne par décision technique importante, au moment où elle est prise. Ce journal alimente directement la partie « choix techniques » du rapport.

Date	Décision	Alternatives écartées	Raison
	Riverpod pour les états	setState, Provider, Bloc	AsyncValue natif, injection testable, état hors widgets
	SQLite pour toute la persistance	shared_preferences, Hive	Requêtes sur des historiques datés, séparation nette utilisateur et cache
	Catalogue coté en dollars	Mélange euros et dollars	Effet de change visible partout, moins d'incertitude sur le plan gratuit